

Memory Model — TOÀN BỘ qua HÌNH VẼ

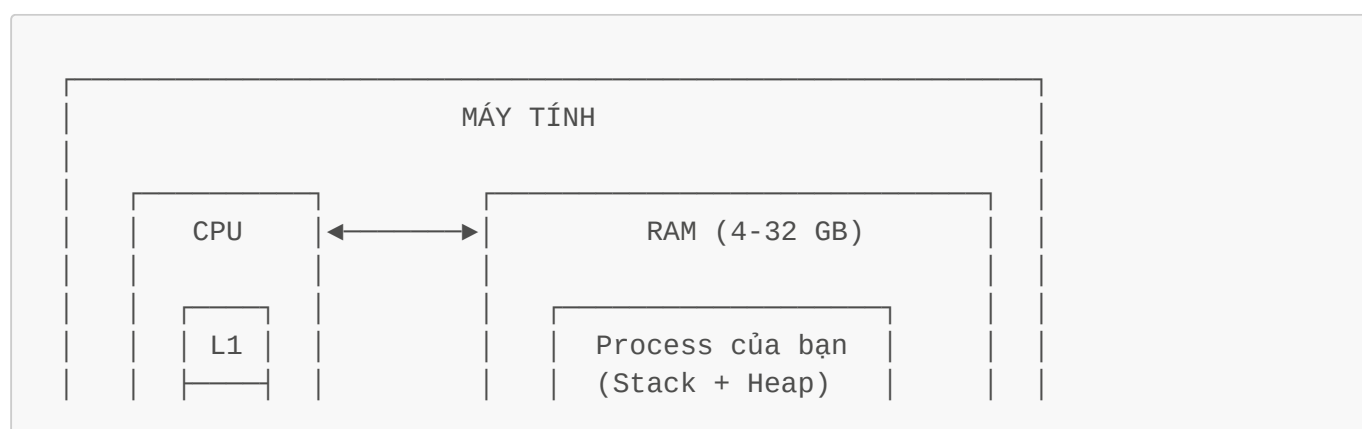
File này là **companion visual** cho `memory-model.md`. Mọi khái niệm đều được vẽ ra. Đọc từ trên xuống dưới — mỗi phần xây trên phần trước.

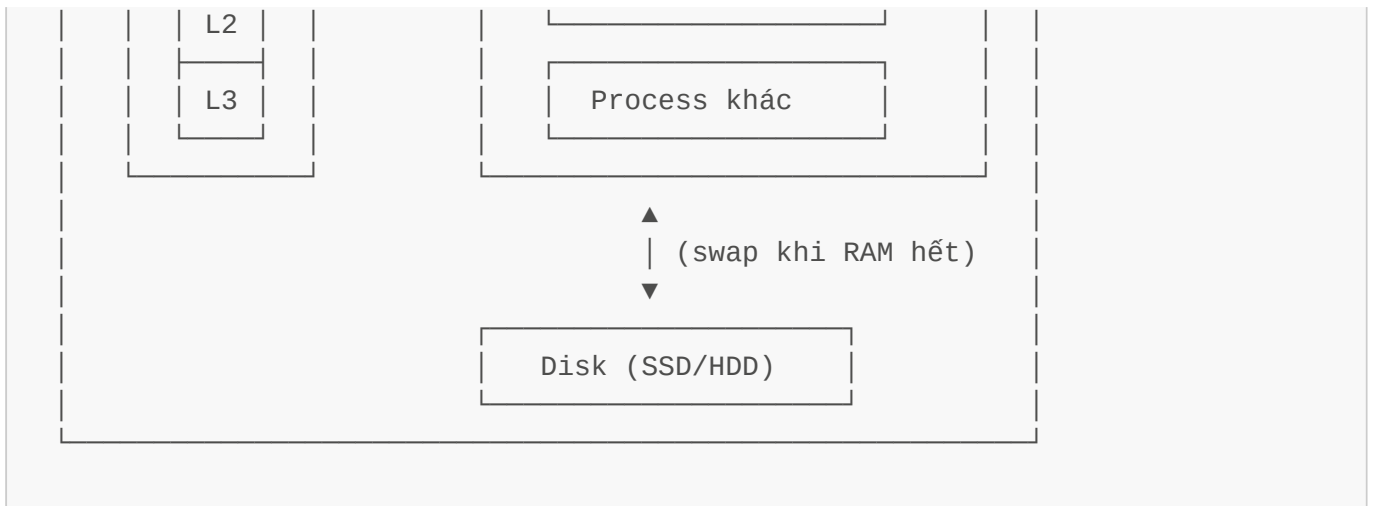
Mục lục hình minh họa

1. Bức tranh lớn: Máy tính nhìn từ trên xuống
2. Hệ phân cấp bộ nhớ (Memory Hierarchy)
3. Virtual Memory — Ảo hóa địa chỉ
4. Process Address Space — 6 vùng
5. Stack — cách hoạt động
6. Heap — cách hoạt động
7. Stack vs Heap — so sánh trực quan
8. Bộ nhớ của các kiểu Rust
9. String / &str / Vec — fat pointer
10. Box / Rc / Arc
11. Alignment & Padding — căn lề bộ nhớ
12. Niche Optimization — Option nén
13. CPU Cache — L1/L2/L3
14. Cache Line & False Sharing
15. Cache Associativity
16. Atomic & CAS
17. Memory Ordering — Reordering
18. Async State Machine
19. Lock-Free: Treiber Stack
20. ABA Problem
21. NUMA
22. Allocators — glibc/jemalloc/mimalloc
23. Memory-Mapped Files (mmap)

1. Bức tranh lớn

Khi bạn chạy `cargo run` → một **process** ra đời. Process này có:





Ý chính: CPU **chỉ làm việc** trên thanh ghi (register) bên trong nó. Mọi dữ liệu **phải được kéo từ RAM về CPU**.
Càng xa CPU → càng chậm.

2. Hệ phân cấp bộ nhớ

Tốc độ thực tế khi CPU lấy 1 byte dữ liệu:



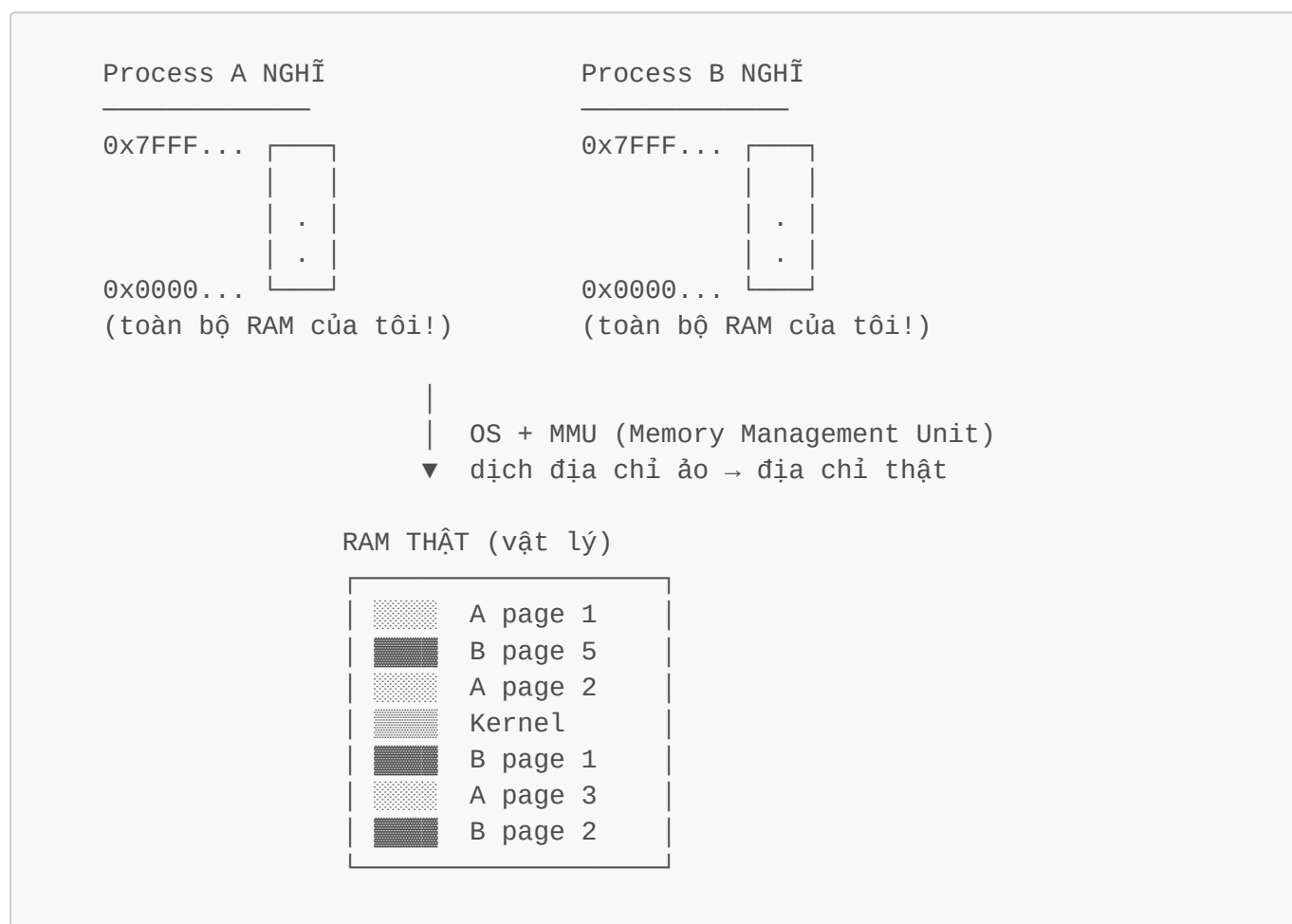
Hình dung số học: nếu L1 = 1 giây, thì:

- L2 = 4 giây
- L3 = 12 giây
- RAM = **100 giây** (~2 phút)
- SSD = **100,000 giây** (~28 giờ)
- HDD = **10,000,000 giây** (~115 ngày)

→ Đây là lý do **cache miss** là kẻ thù của hiệu năng.

3. Virtual Memory

Mỗi process **NGHĨ** rằng nó sở hữu toàn bộ RAM. Sự thật: OS lừa nó.



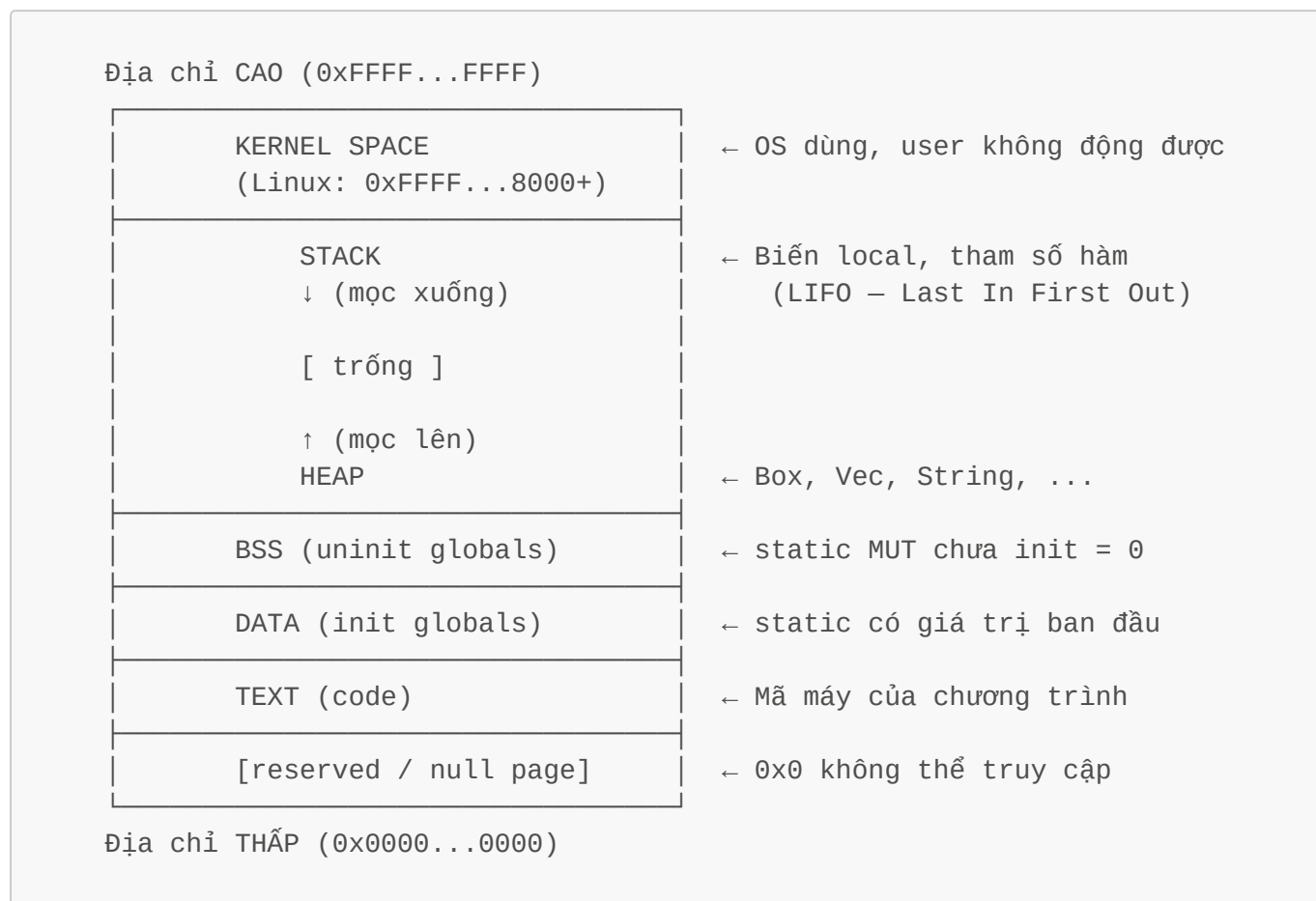
Bảng dịch (Page Table):

Process A:			Process B:		
Virtual	→	Physical	Virtual	→	Physical
0x1000	→	0x8000	0x1000	→	0xA000
0x2000	→	0xC000	0x2000	→	0xB000
0x3000	→	0xE000	0x3000	→	0xD000

→ Cùng địa chỉ ảo **0x1000** nhưng trở về 2 vùng RAM khác nhau → **cô lập process**.

4. Process Address Space

Khi process chạy, không gian địa chỉ ảo được chia thành 6 vùng:

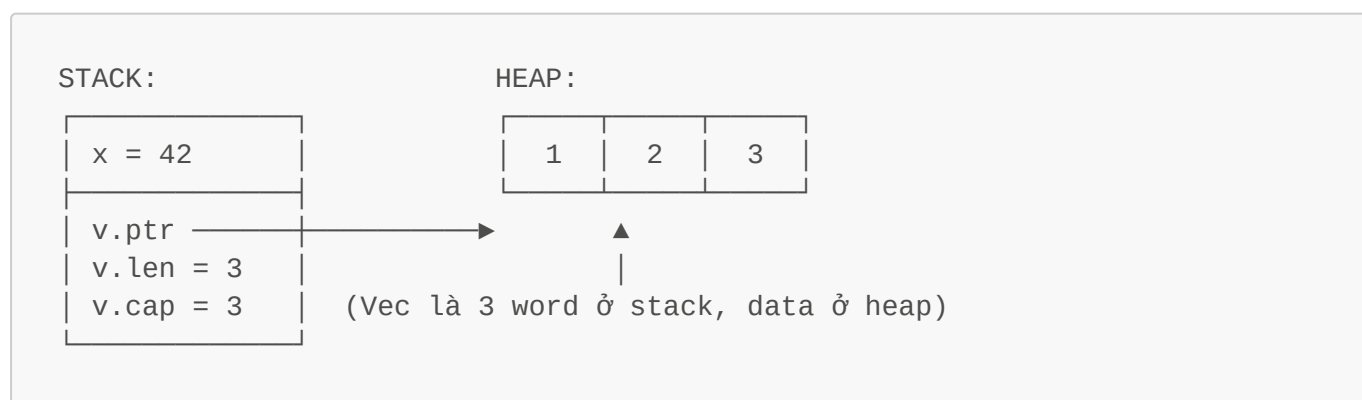


Ví dụ thực tế với code Rust:

```
static MAX: i32 = 100;           // DATA segment
static mut COUNTER: i32 = 0;    // BSS segment

fn main() {                     // TEXT segment (code)
    let x = 42;                 // STACK
    let v = vec![1, 2, 3];      // v (3 word) ở STACK,
                                // [1,2,3] ở HEAP
}
```

Visualize:



5. Stack

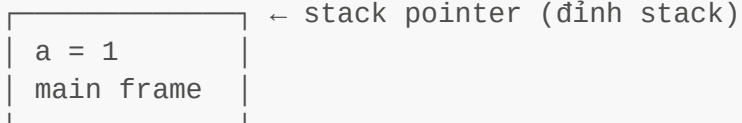
Stack giống **chồng đĩa**: chỉ thêm/bớt ở đỉnh.

Mỗi lần gọi hàm = đẩy 1 "frame" mới lên stack

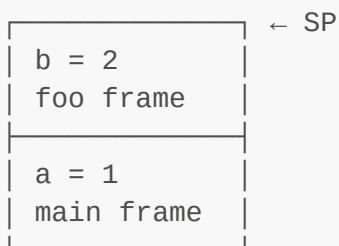
```
fn main() {
    let a = 1;
    foo();
}
fn foo() {
    let b = 2;
    bar();
}
fn bar() {
    let c = 3;
}
```

Diễn biến:

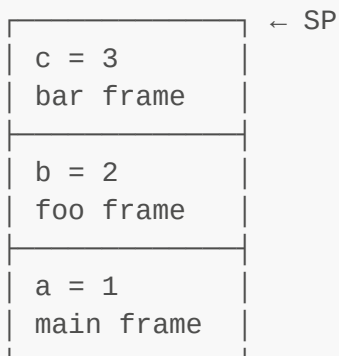
Bước 1: main() chạy



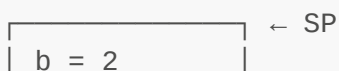
Bước 2: foo() được gọi → đẩy frame mới lên

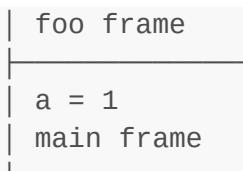


Bước 3: bar() được gọi



Bước 4: bar() return → bar frame BIẾN MẤT (chỉ giảm SP)



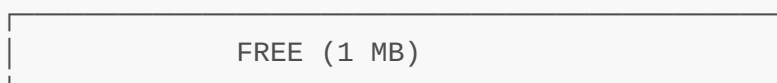


Lý do stack nhanh: chỉ tăng/giảm 1 thanh ghi (Stack Pointer). Không cần tìm chỗ trống.

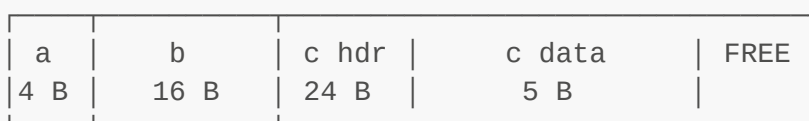
6. Heap

Heap là **vùng nhớ tự do** — bạn xin (`malloc/Box::new`), bạn trả (`free/drop`).

Lúc đầu: heap toàn là bộ nhớ trống

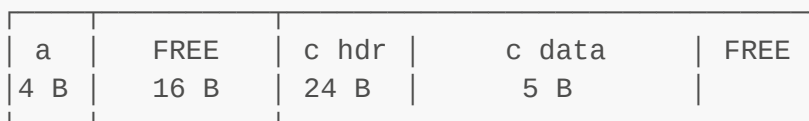


Sau khi: `let a = Box::new(100i32);` // 4 bytes
`let b = Box::new([0u8; 16]);` // 16 bytes
`let c = Box::new("hello".to_string());` // 24 byte (header) + 5 byte (data)



▲ ▲ ▲
 | | |
 ptr ptr ptr ← các biến trên stack trỏ tới đây

Sau khi `drop(b)`:



↑ Lỗ trống – gọi là **FRAGMENTATION**

Allocator quản lý các "lỗ trống" bằng Free List

Free List (danh sách các block trống):

[16 B] → [32 B] → [128 B] → NULL

Khi xin 20 B:

- Bỏ qua 16 B (quá nhỏ)
- Lấy 32 B → cắt 20 B để dùng, trả 12 B vào free list

Sau:

[16 B] → [12 B] → [128 B] → NULL

7. Stack vs Heap

STACK

Tốc độ:	⚡⚡⚡
Quản lý:	Tự động
Kích thước:	Cố định (compile)
Lifetime:	Frame scope
Capacity:	Nhỏ (8 MB)
Thread:	Riêng /thread

HEAP

Tốc độ:	🐢
Quản lý:	Thủ công
Kích thước:	Động (runtime)
Lifetime:	Đến khi free
Capacity:	Lớn (vài GB)
Thread:	Chia sẻ (cần sync)

Đặt biến lên STACK khi:

- Kích thước biết lúc compile
- Sống trong 1 scope
- Nhỏ (vài bytes → vài KB)
- Sao chép (Copy)

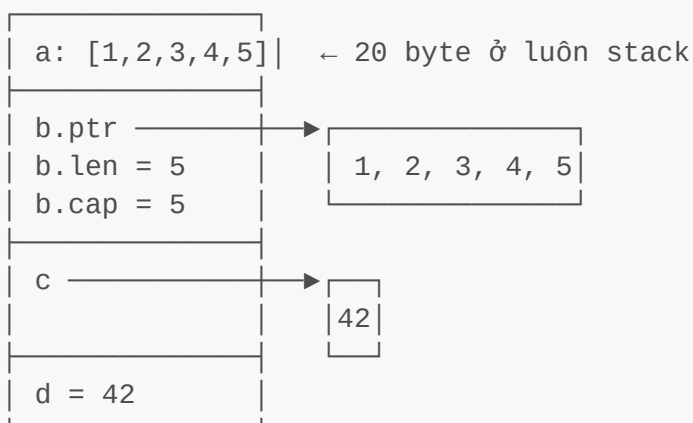
Đặt biến lên HEAP khi:

- Kích thước biết lúc runtime
- Sống lâu hơn scope
- Lớn (MB → GB)
- Chuyển sở hữu (Move)

Ví dụ ranh giới:

```
let a: [i32; 5] = [1,2,3,4,5]; // STACK (size cố định)
let b: Vec<i32> = vec![1,2,3,4,5]; // Vec header ở STACK, data ở HEAP
let c: Box<i32> = Box::new(42); // c ở STACK, 42 ở HEAP
let d: i32 = 42; // STACK
```

STACK:



HEAP:

8. Bộ nhớ các kiểu Rust

8.1. Kiểu nguyên thủy

i8	1 byte		
i16	2 bytes		
i32	4 bytes		
i64	8 bytes		
i128	16 bytes		
f32	4 bytes		(IEEE 754 single)
f64	8 bytes		(IEEE 754 double)
bool	1 byte		(chỉ dùng 1 bit, lãng phí 7 bit)
char	4 bytes		(Unicode scalar value)
usize	8 bytes (trên 64-bit)		
isize	8 bytes (trên 64-bit)		

8.2. Tuple

```
let t: (i32, f64, bool) = (1, 2.0, true);
```

STACK:

i32: 1	4 bytes
[padding 4 byte]	← để f64 căn lề 8
f64: 2.0	8 bytes
bool: true	1 byte
[padding 7 byte]	← để toàn tuple căn lề 8

Tổng: 24 bytes (không phải 4+8+1=13!)

8.3. Array

```
let arr: [i32; 5] = [10, 20, 30, 40, 50];
```

STACK:

10	20	30	40	50	← 5 × 4 = 20 bytes liên tiếp
----	----	----	----	----	------------------------------



▲
└ arr là pointer ẨN tới đây

8.4. Struct

```
struct Point { x: i32, y: i32 }
let p = Point { x: 10, y: 20 };
```

STACK:

x = 10	y = 20	8 bytes (2 × 4)
--------	--------	-----------------

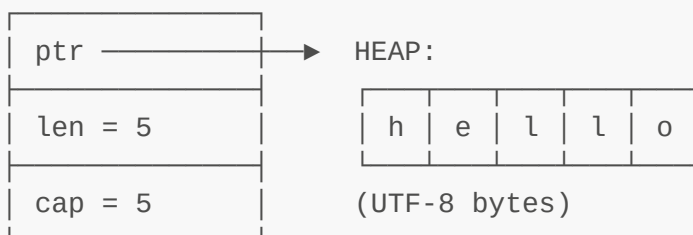
9. Fat Pointer

`String`, `&str`, `Vec<T>`, `&[T]` đều là **fat pointer** = pointer + metadata.

9.1. String

```
let s = String::from("hello");
```

STACK (24 bytes – fat pointer):



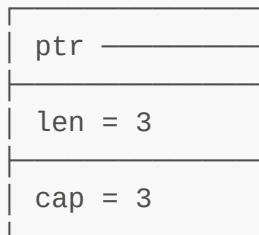
- **ptr** (8 byte): trỏ tới byte đầu tiên trên heap
- **len** (8 byte): có 5 byte đang dùng
- **cap** (8 byte): tổng dung lượng đã cấp phát

9.2. Vec<T>

Giống y hệt String, chỉ là T tổng quát:

```
let v: Vec<i32> = vec![1, 2, 3];
```

STACK (24 bytes):



HEAP:



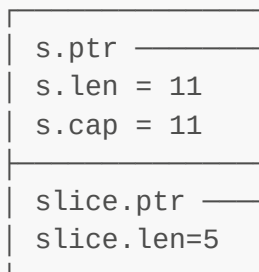
(3 × 4 = 12 bytes)

9.3. &str

&str là **slice** — chỉ ptr + len (không cap, vì không sở hữu).

```
let s = String::from("hello world");
let slice: &str = &s[0..5];    // "hello"
```

STACK:



HEAP:



s)

└ s.ptr

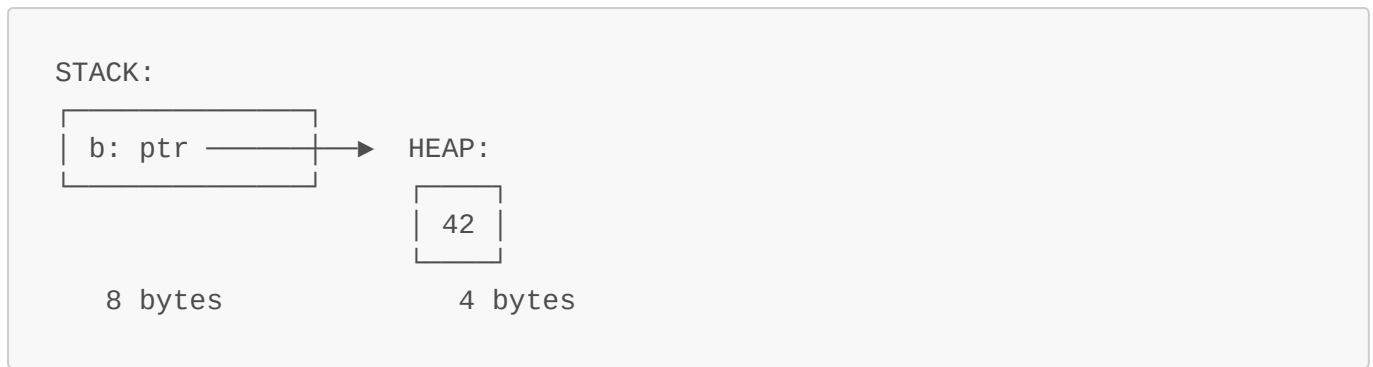
└ slice trở vào ĐÂY luôn (cùng heap với

→ **&str không** sao chép data, chỉ là "cửa sổ" nhìn vào string khác.

10. Box, Rc, Arc

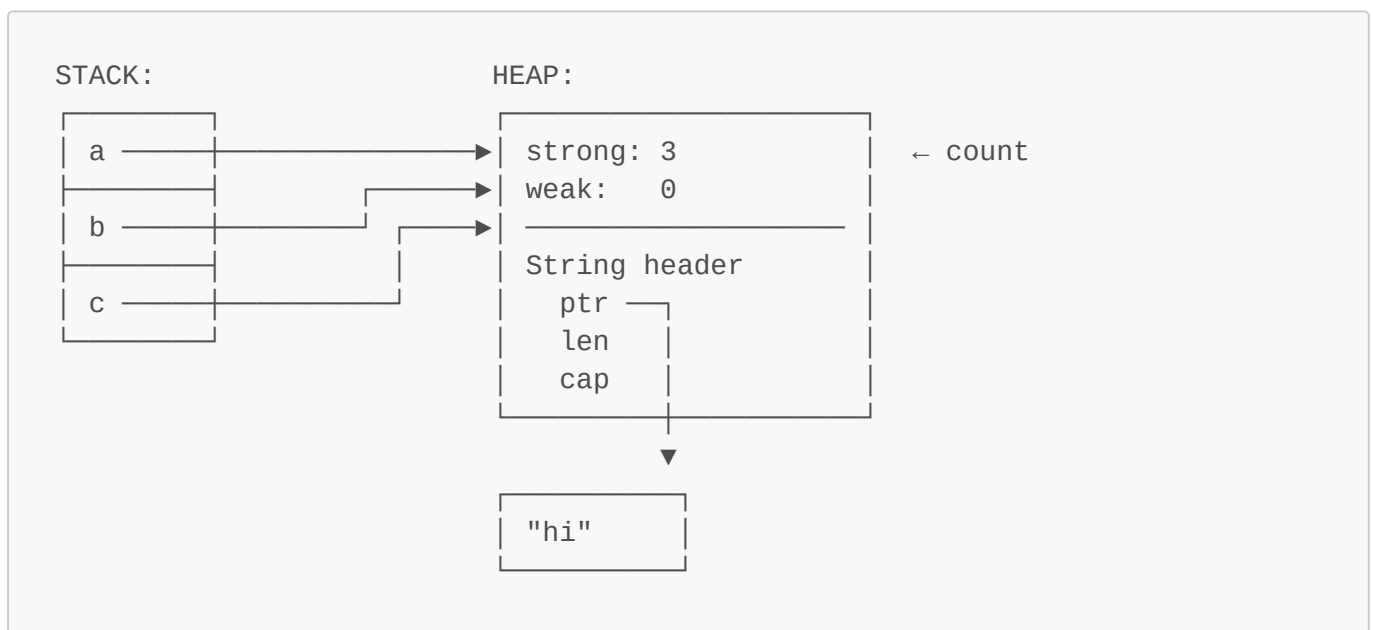
10.1. Box<T> — đơn sở hữu

```
let b: Box<i32> = Box::new(42);
```



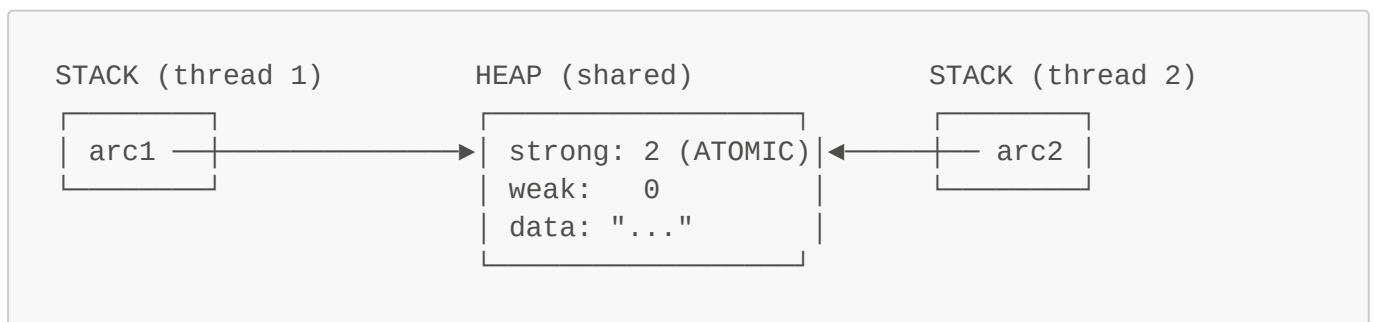
10.2. Rc<T> — đếm tham chiếu (single-thread)

```
let a: Rc<String> = Rc::new(String::from("hi"));
let b = Rc::clone(&a); // không clone data, chỉ tăng count
let c = Rc::clone(&a);
```



Khi **a** drop → strong = 2. Khi cả 3 drop → strong = 0 → free.

10.3. Arc<T> — Rc nhưng cho multi-thread



Khác biệt với Rc: counter dùng **atomic** (**AtomicUsize**) để tăng/giảm an toàn từ nhiều thread.

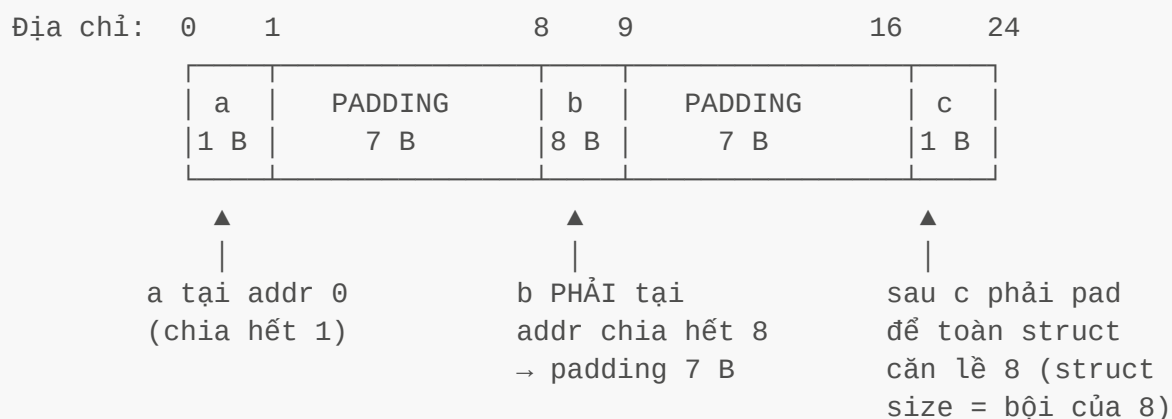
→ Arc chậm hơn Rc một chút vì atomic ops, nhưng thread-safe.

11. Alignment & Padding

CPU đọc bộ nhớ theo "khúc" (8/16/32 bytes). Dữ liệu **phải bắt đầu** ở địa chỉ chia hết cho size của nó.

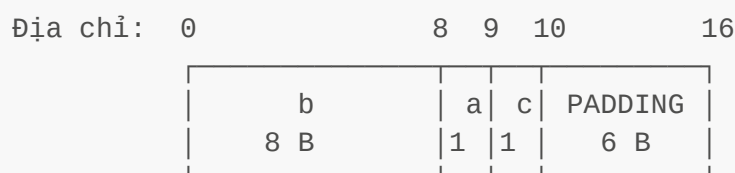
Quy tắc: type có size N → phải đặt ở địa chỉ chia hết cho N

```
struct Bad {
    a: u8,    // 1 byte
    b: u64,   // 8 bytes
    c: u8,    // 1 byte
}
// Bạn nghĩ: 1+8+1 = 10 bytes?
// Thực tế: 24 bytes!
```



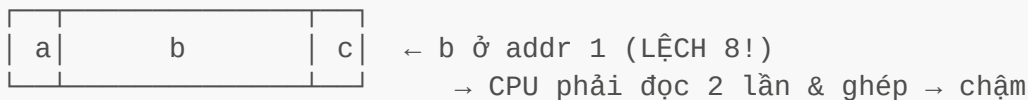
Sắp xếp lại field từ LỚN → NHỎ → tiết kiệm

```
struct Good {
    b: u64,    // 8 bytes
    a: u8,    // 1 byte
    c: u8,    // 1 byte
}
// Size: 16 bytes (tiết kiệm 8 byte!)
```



Hoặc dùng `#[repr(packed)]` để bỏ padding (CHẬM hơn!)

```
#[repr(packed)]
struct Packed { a: u8, b: u64, c: u8 } // 10 bytes
```



12. Niche Optimization

Rust phát hiện "giá trị bất hợp lệ" để **nén** `Option`.

Bài toán: `Option<bool>` cần bao nhiêu byte?

`bool` chỉ dùng 2 giá trị: 0 và 1. Còn 254 giá trị "trống". → Rust dùng **2** để biểu diễn `None`!

```
Option<bool>: 1 byte
Some(false)  = 0x00
Some(true)   = 0x01
None         = 0x02 ← niche!
```

`Option<&T>` cũng vậy

Reference `&T` **không bao giờ** là null (`0x0`). Rust dùng `0x0` để biểu diễn `None`.

```
Option<&i32>: 8 bytes (KHÔNG phải 16!)

Some(&x):
    0x7FFF...1234 ← địa chỉ thật

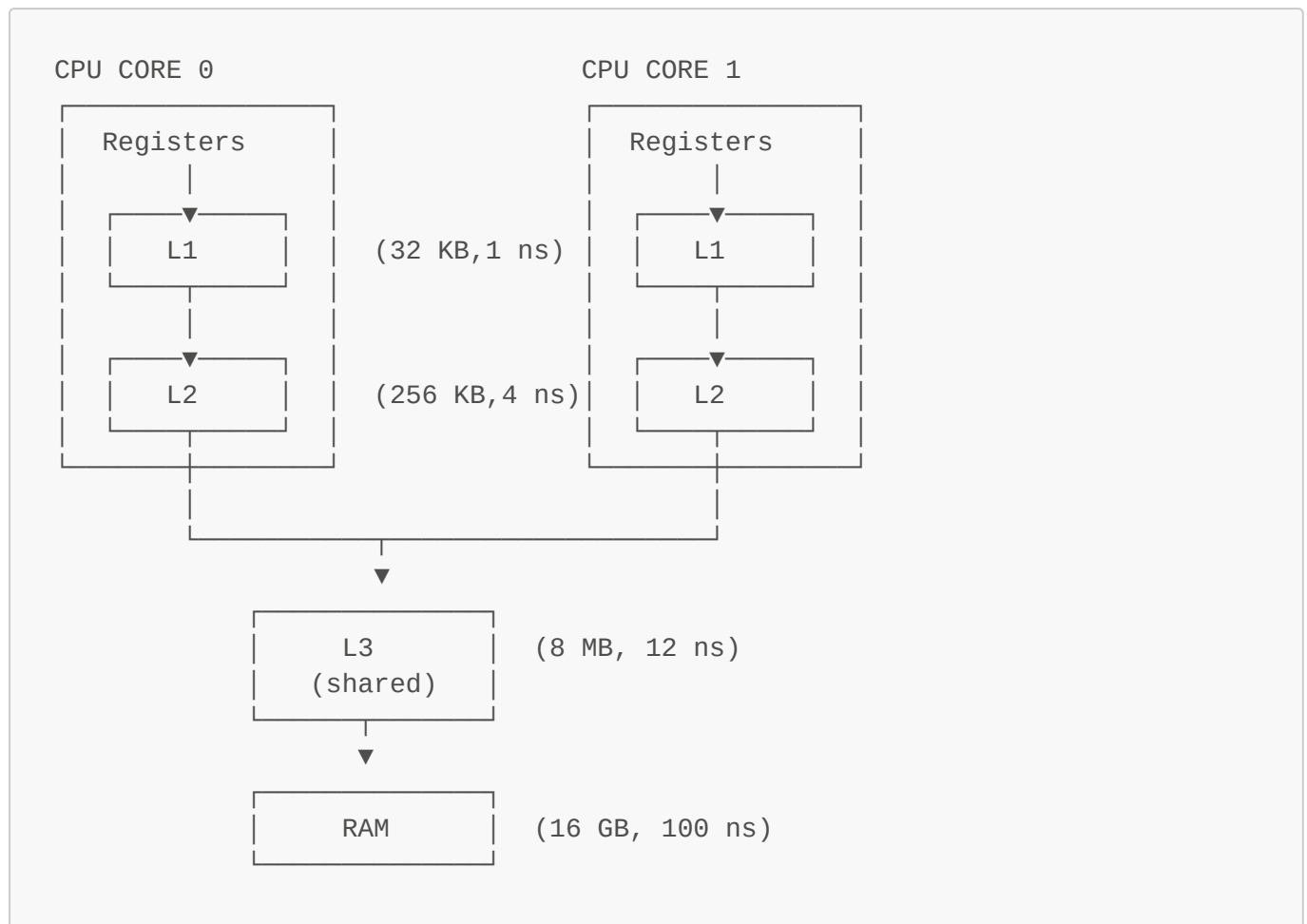
None:
    0x0000...0000 ← niche (sẽ không xung đột!)
```

So với C, `Option<int*>` cần thêm 1 bool tag → 16 bytes:

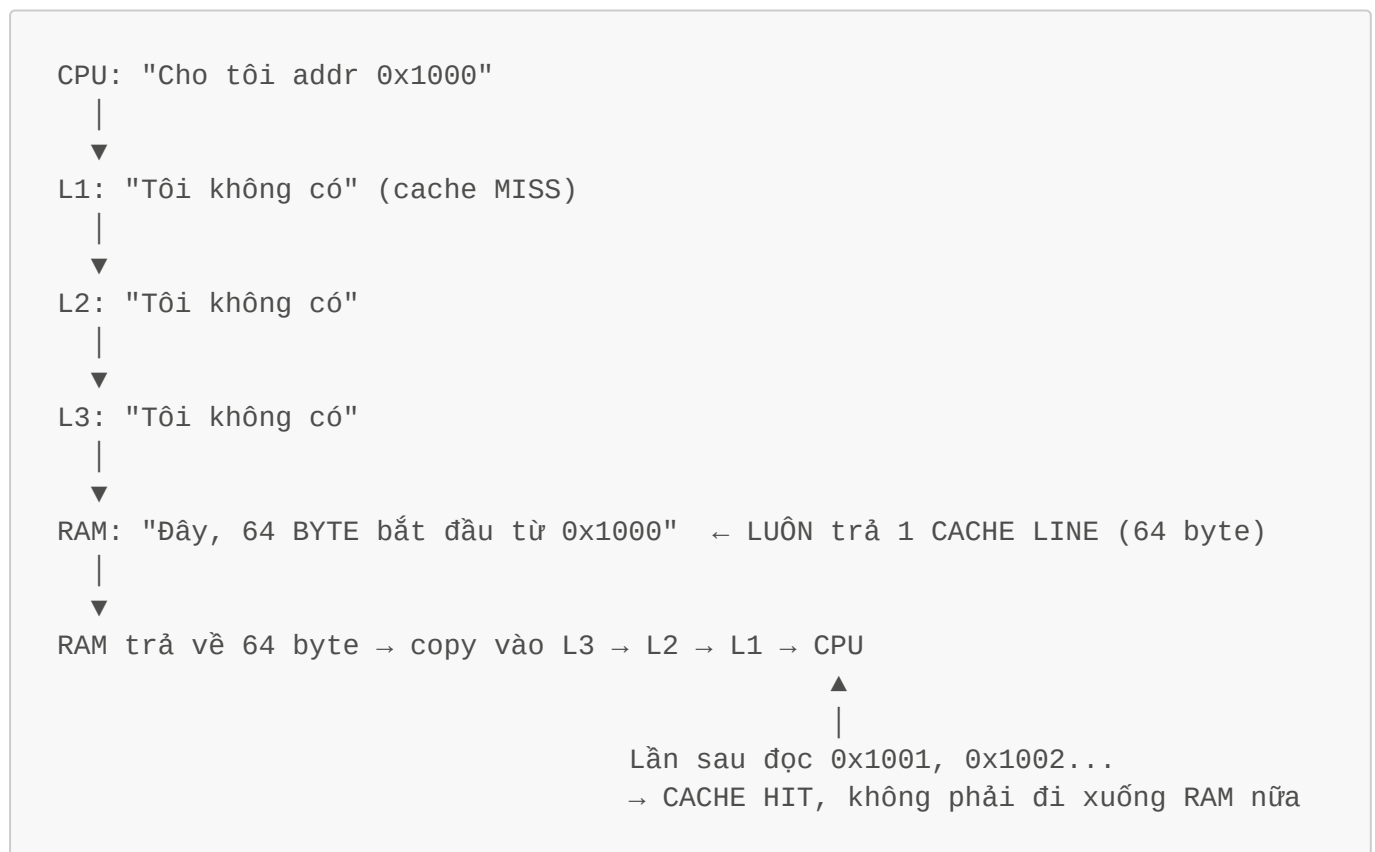
```
C struct {bool has_value; int* ptr;} → 16 bytes (lãng phí 8 byte!)
Rust Option<&i32>                    → 8 bytes
```

13. CPU Cache

CPU không đọc thẳng từ RAM. Nó kéo dữ liệu qua **3 cấp cache** trước:



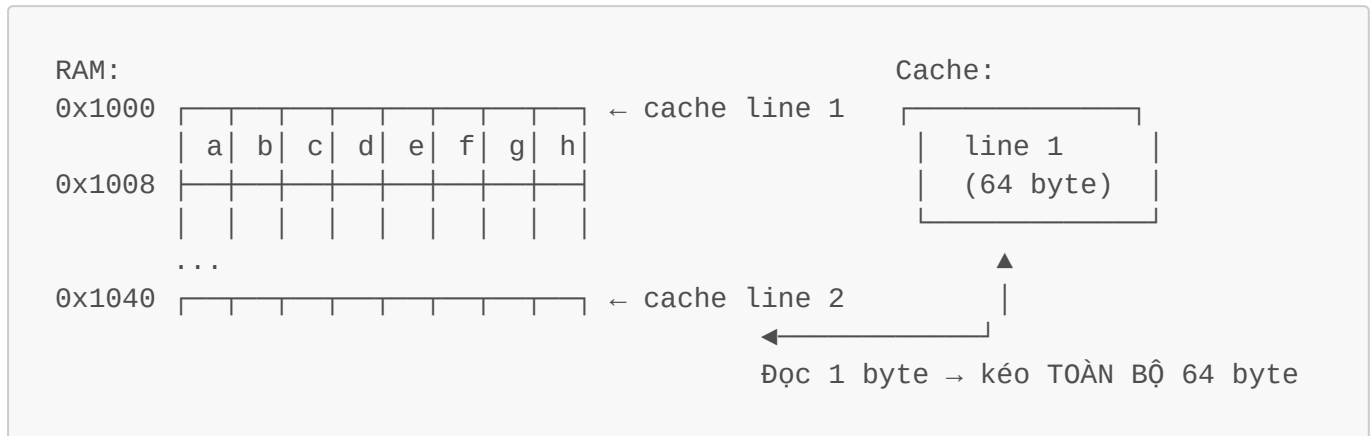
Khi CPU đọc `arr[0]`



→ Đây là lý do **iteration tuần tự** nhanh hơn random access nhiều lần.

14. Cache Line & False Sharing

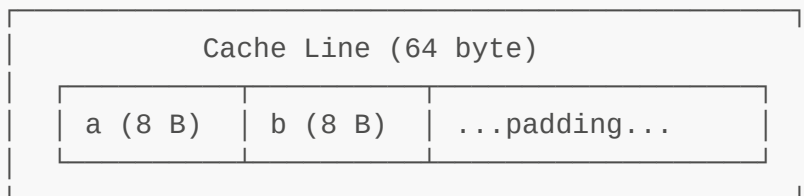
Cache line = đơn vị chuyển dữ liệu giữa RAM ↔ Cache (64 byte)



False Sharing — bất đa luồng

```
struct Counters {
    a: AtomicU64, // Thread 1 cập nhật
    b: AtomicU64, // Thread 2 cập nhật
}
```

Cả a và b nằm trong CÙNG 1 cache line 64 byte:



Thread 1 (core 0):

- Ghi vào a
 - Core 0 lấy cache line về L1
 - Đánh dấu "modified"
 - Lại muốn cập nhật a
 - Yêu cầu lại từ core 1...
- PING-PONG vô tận

Thread 2 (core 1):

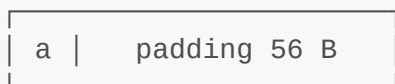
- Ghi vào b
 - Core 1 muốn cache line này
 - Báo core 0: "đưa tao!"
 - Core 0 phải FLUSH về L3
 - Core 1 kéo về
 - Cập nhật b
 - Lại flush về...
- PING-PONG vô tận

Hiệu năng: chậm 10-100x so với khi tách riêng cache line!

Khắc phục: padding để 2 biến ở 2 cache line khác nhau

```
struct Counters {
    a: AtomicU64,
    _pad: [u8; 56], // đẩy b sang cache line khác
    b: AtomicU64,
}
```

Cache Line 1:



Thread 1 dùng
(không ảnh hưởng đến thread 2)

Cache Line 2:



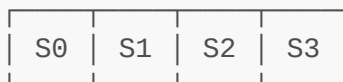
Thread 2 dùng

15. Cache Associativity

Cache là **mảng nhỏ** → không thể chứa địa chỉ tùy ý → cần map.

Direct-Mapped (1-way) — mỗi addr chỉ có 1 chỗ duy nhất

Cache (4 slot):



Addr → slot: $\text{hash} = \text{addr} \% 4$

addr 0 → S0

addr 4 → S0 ← CONFLICT với addr 0!

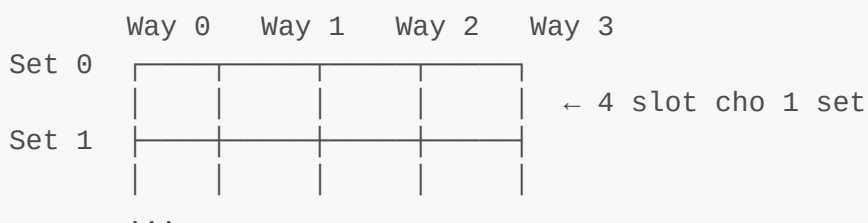
addr 8 → S0 ← CONFLICT!

addr 12 → S0 ← CONFLICT!

Nếu vòng lặp truy cập 0, 4, 0, 4, 0, 4... → liên tục evict → CHẬM

Set-Associative (4-way) — mỗi addr có 4 chỗ chọn

Cache (4 set × 4 way):



Addr → set: $\text{hash} = (\text{addr} / 64) \% \text{num_sets}$

→ Cùng 1 set có thể chứa 4 line khác nhau → ít conflict hơn

Bẫy: Power-of-2 strides

```
// 1024 = power of 2 → có thể xung đột set
let big: [[f64; 1024]; 1024] = ...;
for j in 0..1024 {
    sum += big[j][0]; // mỗi truy cập cách nhau 1024*8 = 8192 B
}                      // → cùng 1 set → conflict!
```

Fix: pad lên 1025 (số lẻ) → các stride khác nhau → trải đều các set.

16. Atomic & CAS

Vấn đề: 2 thread cùng tăng 1 biến

```
let mut counter = 0;
// Thread 1: counter += 1;
// Thread 2: counter += 1;
```

Trên CPU, `counter += 1` thực ra là **3 lệnh**:

1. LOAD counter → register (đọc giá trị từ RAM)
2. ADD register, 1 (cộng 1)
3. STORE register → counter (ghi lại vào RAM)

Race condition

counter = 0 ban đầu

Thread 1

LOAD counter (= 0)

ADD 1 (=1)

STORE counter (= 1)

Thread 2

LOAD counter (= 0)

ADD 1 (=1)

STORE counter (= 1)

Kết quả: counter = 1 (đáng lẽ phải = 2!)

Giải pháp: Atomic — lock cache line trong khi đọc-sửa-ghi

```
use std::sync::atomic::{AtomicI64, Ordering};
let counter = AtomicI64::new(0);
counter.fetch_add(1, Ordering::SeqCst);
```

Lệnh CPU: LOCK XADD → **atomic** (không thể bị chia cắt)

Thread 1

 LOCK cache line

LOAD counter (= 0)

ADD 1

STORE counter (= 1)

 UNLOCK

Thread 2

|| Chờ...

 LOCK

LOAD counter (= 1)

ADD 1

STORE counter (= 2)

 UNLOCK

Kết quả: counter = 2 ✓

CAS (Compare-And-Swap) — vũ khí của lock-free

CAS(addr, expected, new):

- Nếu `*addr == expected` → ghi `new` → trả `true`
- Nếu không → không làm gì → trả `false`

CAS atomically:

```

    if (*addr == expected) {
        *addr = new;
        return true;
    } else {
        return false;
    }

```

Toàn bộ trong 1 cycle, không bị chia cắt!

Pattern: retry loop

```

loop {
    let cur = counter.load();           // 1. Đọc giá trị hiện tại
    let new = cur + 1;                  // 2. Tính giá trị mới
    if counter.cas(cur, new) {          // 3. Thử cập nhật
        break;                          //   Thành công → thoát
    }                                  //   Thất bại → quay lại 1
}

```

Thread A:
 cur = 5
 new = 6
 CAS(5, 6) ✓

Thread B:
 cur = 5
 new = 6
 CAS(5, 6) ✗ (vì counter giờ = 6, không = 5)
 cur = 6 (load lại)
 new = 7
 CAS(6, 7) ✓

17. Memory Ordering

CPU và compiler **tự ý sắp xếp lại** lệnh để tối ưu! Điều này gây bug trong đa luồng.

Ví dụ bug

```

// Thread 1:
data = 42;
ready = true;

// Thread 2:
while !ready {}
println!("{}", data);

```

Bạn nghĩ: thread 2 sẽ in ra 42.

Thực tế: CPU có thể đảo `data = 42` và `ready = true`!

Thực thi đảo ngược của thread 1:

```
ready = true;      ← thread 2 thấy ready=true ngay!
data  = 42;        ← nhưng data vẫn = 0 lúc đó
```

→ Thread 2 in ra 0 hoặc rác.

Sơ đồ Memory Ordering

Relaxed

- Chỉ atomic, không ràng buộc thứ tự
- Nhanh nhất
- Dùng: counter đếm thuần túy

```
Thread 1: write A (relaxed) | Thread 2: read A
           write B (relaxed) |           read B
```

Có thể thấy: A trước B, B trước A, không A, không B...

Acquire / Release

- Release: tất cả store TRƯỚC nó phải hiện thực hóa
- Acquire: tất cả load SAU nó thấy được các store đó
- Cặp đôi: 1 thread Release, 1 thread Acquire

```
Thread 1:      Thread 2:
data = 42;      while !ready.load(Acquire) {}
ready.store(true,Release); // ↑ Sau dòng này, data CHẮC=42
```

SeqCst (mạnh nhất)

- Tất cả thread thấy CÙNG 1 thứ tự toàn cục
- Như có 1 "đồng hồ thế giới"
- Chậm nhất
- Dùng khi không chắc → an toàn

Visualize Acquire-Release

Thread 1:

```
data = 42;
data2 = 99;
```



```
ready.store(true, Release);
```

Thread 2:

(Acquire BLOCKS đến khi

```

| ready = true)
▼
while !ready.load(Acquire) {}
    ←
// Bây giờ data = 42 GUARANTEED
// và data2 = 99 GUARANTEED
println!("{}", data, data2);

```

Mọi write TRƯỚC Release $\implies$ Mọi read SAU Acquire

18. Async State Machine

async fn được compiler biến thành **enum state machine**.

```

async fn task() {
    let a = step1().await;
    let b = step2(a).await;
    println!("{}", b);
}

```

Compiler tạo:

```

enum TaskState {
    Start,
    AfterStep1 { a: i32, fut: Step2Future },
    Done,
}

```

Diễn biến

Bước 1: tạo Future

```
TaskState::Start
```

Bước 2: executor poll lần 1

- bắt đầu step1()
- step1 chưa xong (vd: chờ IO)
- state machine LƯU LẠI vị trí:

```

TaskState::AfterStep1 {
    a: <pending>
    fut: <step1 future>
}

```

Bước 3: IO xong → executor poll lần 2
 → resume từ AfterStep1
 → lấy a, gọi step2(a)
 → step2 cũng chưa xong → lưu vị trí
 → ...

Bước cuối:

TaskState::Done

Stack-less coroutine — lý do async Rust khác Go

GOROUTINE (Go):

Stack riêng
8 KB → 1 MB
(mọc động)

Tốn RAM lớn
Switch tốn
Stack copy

ASYNC TASK (Rust):

Enum state
~vài chục byte
(cố định)

Tiết kiệm RAM cực kỳ
Switch = pop enum
Không stack copy

→ 1 triệu task async Rust = vài chục MB. 1 triệu goroutine = vài GB.

Pin — neo state machine

Một số future có **con trỏ tự tham chiếu** (trỏ vào field khác của chính nó):

self_ref future:

data: "hello"
 ptr: _____

← (trỏ vào data!)

Nếu future **bị di chuyển** sang địa chỉ khác → ptr trỏ vào địa chỉ cũ → DANGLING!

→ `Pin<&mut T>` cấm di chuyển → đảm bảo ptr luôn hợp lệ.

19. Treiber Stack

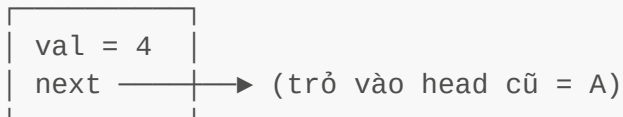
Stack lock-free dùng CAS — không cần Mutex.

Cấu trúc

```
head → [val=3] → [val=2] → [val=1] → null
        (Node A)   (Node B)   (Node C)
```

Push 4

Bước 1: Tạo node mới, next = head hiện tại



new_node

```
head → [3] → [2] → [1] → null
      ▲
      |
new_node.next
```

Bước 2: CAS(head, A, new_node)

- Nếu head VẪN = A → đổi head thành new_node
- Nếu head đã thay đổi → RETRY

Sau khi CAS thành công:

```
head → [4] → [3] → [2] → [1] → null
```

Pop

Bước 1: đọc head = A

Bước 2: lấy A.next = B

Bước 3: CAS(head, A, B)

- Thành công → trả về A
- Thất bại → retry

Trước:

```
head → [3] → [2] → [1] → null
      (A)
```

Sau:

```
head → [2] → [1] → null
      (B)
```

A bị "tách ra" → free

20. ABA Problem

Cái bẫy cổ điển khi dùng CAS với pointer.

Kịch bản

Trạng thái ban đầu:

head → [A] → [B] → null

Thread 1: muốn pop

- Đọc head = ptr_A
- Đọc A.next = ptr_B
- Sắp CAS(head, ptr_A, ptr_B)... ZZZ (bị OS cướp CPU)

Trong lúc đó, Thread 2 cực kỳ tích cực:

- Pop A: head → B
- Pop B: head → null
- Free A
- malloc 1 node mới → TÌNH CỜ allocator trả lại địa chỉ của A
- Đặt val = X, push lên: head → A (cùng địa chỉ) → null

Trạng thái sau:

head → [A'] → null
(cùng địa chỉ A, nhưng KHÁC dữ liệu!)

Thread 1 thức dậy:

- CAS(head, ptr_A, ptr_B)
- head VẪN = ptr_A → CAS THÀNH CÔNG! (sai lầm!)
- head → ptr_B → nhưng ptr_B đã FREE rồi!
- DANGLING POINTER → crash hoặc lỗi nghiêm trọng

Tóm tắt vấn đề:

Thời điểm T1	T2	T3 (thread 1 wake)
head = A	head = A	head = A
thread 1	thread 2	thread 1
đọc A	pop A, B,	CAS thấy A
ngủ thiếp	malloc về A	THÀNH CÔNG (sai!)
▲ Đọc giá trị A	▲ Giá trị đổi A→B→A (nhưng A khác A ban đầu)	▲ "Giá trị không đổi!" (sai!)

Giải pháp 1: Tagged Pointer (counter)

Thay vì lưu chỉ pointer, lưu (pointer, tag):

```
head = (ptr_A, version = 5)
```

Mỗi lần thay đổi, tag++:

```
Pop A: head = (ptr_B, 6)
```

```
Pop B: head = (null, 7)
```

```
Push A: head = (ptr_A, 8)
```

Thread 1 CAS với (ptr_A, 5):

```
head hiện tại = (ptr_A, 8)
```

CAS THẤT BẠI vì tag khác → an toàn!

Giải pháp 2: Hazard Pointer / Epoch (crossbeam)

Trước khi đọc, thread 1 "đăng ký" rằng nó đang dùng A:

Hazard Pointer Table:

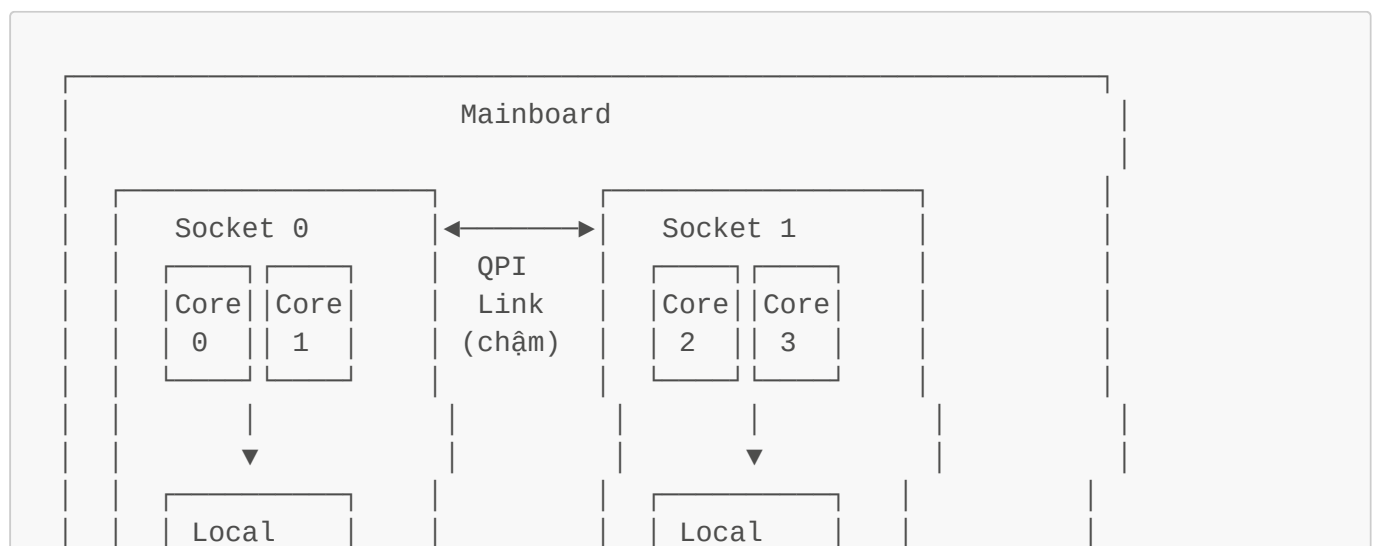
Thread 1 đang giữ: ptr_A	← đăng ký
Thread 2 đang giữ: -	

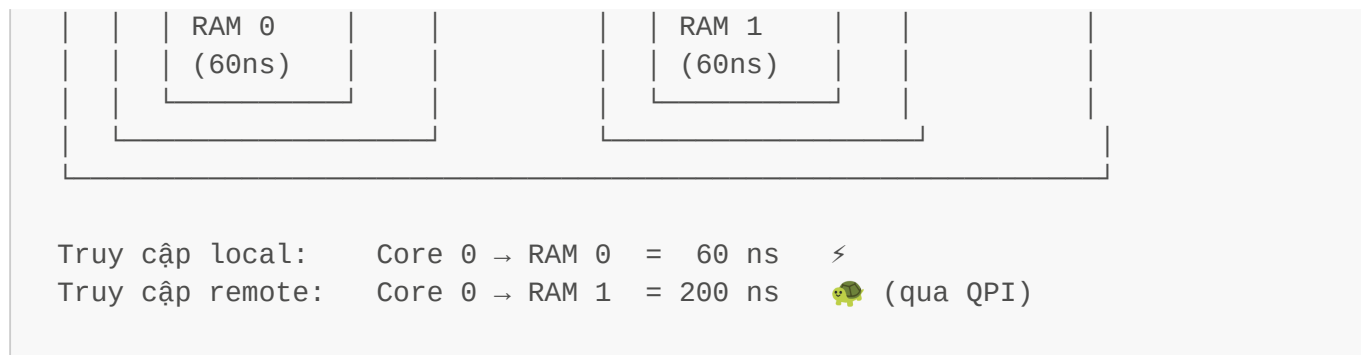
Khi thread 2 muốn free A:

- Kiểm tra HP table
- "Thread 1 đang giữ A → KHÔNG free, đợi đã"
- Đặt vào "retired list", free sau

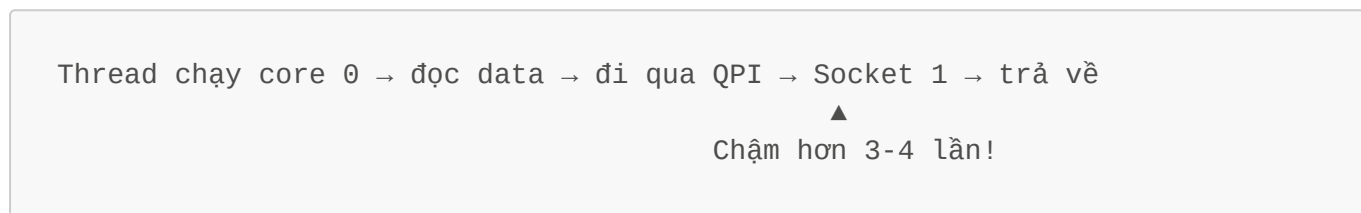
21. NUMA

Trên server lớn, có nhiều **socket** CPU, mỗi socket có RAM riêng.





Bây: thread chạy ở Socket 0 nhưng data ở Socket 1



Fix: NUMA-aware allocation

```
// numactl --cpunodebind=0 --membind=0 ./my_program
// → Thread & data đều ở Socket 0
```

```
First-touch policy:
- malloc 1 GB → chưa cấp RAM thật
- Thread X (chạy Socket 1) ghi vào → Linux cấp RAM ở Socket 1
- Thread X đọc rất nhanh

→ "Người đầu tiên chạm vào memory" quyết định nó nằm ở socket nào
```

22. Allocators

glibc malloc (mặc định Linux)

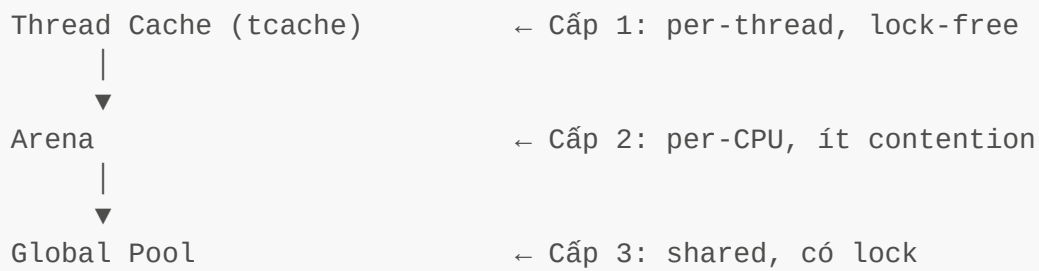
```
Mỗi thread có "arena" riêng (giảm contention):

Thread 1 → Arena 1 → [bin 16] [bin 32] [bin 64]...
Thread 2 → Arena 2 → [bin 16] [bin 32] [bin 64]...
Thread 3 → Arena 3 → [bin 16] [bin 32] [bin 64]...

Vấn đề: fragmentation tệ với pattern alloc/free phức tạp
```

jemalloc (Facebook, Rust trước đây)

Hierarchy 3 tầng:



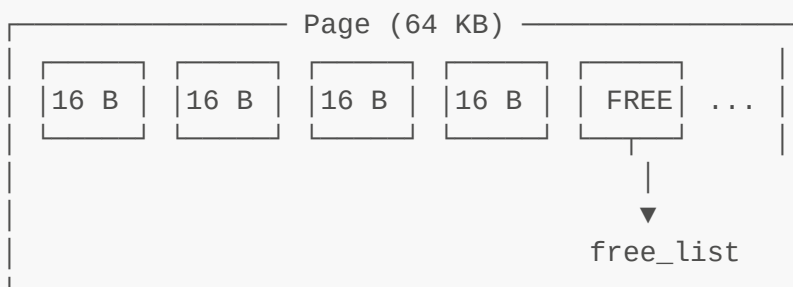
Mỗi cấp có bins theo size class:

bin 8B, 16B, 24B, 32B, 48B, 64B, 80B, 96B, 112B, 128B...
(~150 size classes)

→ Phân mảnh thấp, tốt cho server chạy lâu dài

mimalloc (Microsoft)

Free list theo "page" 64 KB:



Mỗi page chỉ 1 size class → alloc cực nhanh

Khoảng 7-25% nhanh hơn jemalloc trong nhiều workload

So sánh ngắn gọn

	<u>glibc</u>	<u>jemalloc</u>	<u>mimalloc</u>	<u>tcmalloc</u>
Mặc định:	✓	-	-	-
Server lâu dài: Δ frag		✓✓✓	✓✓	✓✓
Tốc độ:	★	★★	★★★	★★★
Multi-thread:	★★	★★★	★★★	★★★
Memory low:	★	★★	★★★	★★
Profiling:	-	✓ (hữu ích)	-	-

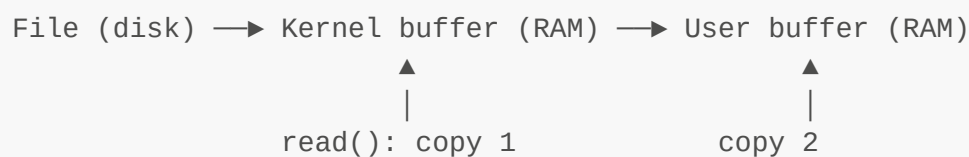
Khuyến nghị cho Big Data:

- Server 24/7 → jemalloc
- Tốc độ cao nhất → mimalloc
- Google ecosystem → tcmalloc

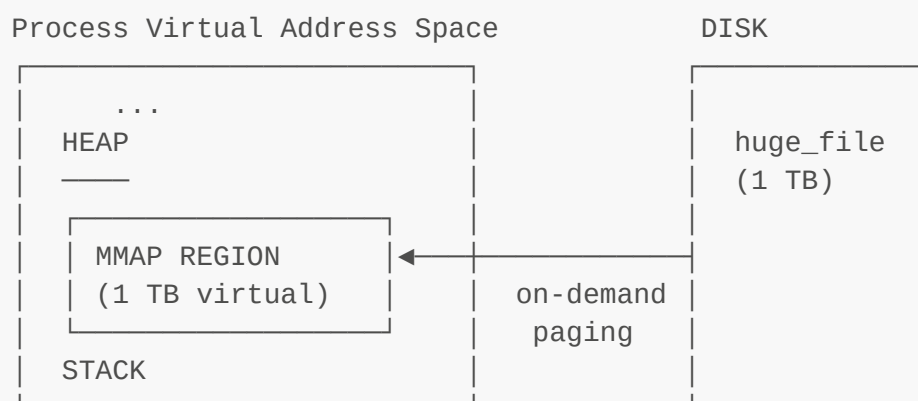
23. mmap

Memory-Mapped Files — ánh xạ file vào không gian địa chỉ ảo, **không** copy vào RAM.

Đọc file thông thường (copy 2 lần!)



mmap (chỉ "trở" tới file)



Process thấy: 1 TB byte trong RAM

Sự thật: OS chỉ kéo PAGE (4 KB) cần thiết về RAM khi truy cập

Demand paging

Bước 1: mmap file 1 TB

→ Không tốn 1 byte RAM nào! Chỉ tạo entries trong page table.

Bước 2: Truy cập byte tại offset 500_000_000_000

→ Page fault!

→ OS đọc 4 KB từ disk vào RAM

→ Map vào virtual address

→ Code tiếp tục

Bước 3: Truy cập byte gần đó (offset 500_000_000_100)

→ Cùng page 4 KB → đã có trong RAM → siêu nhanh

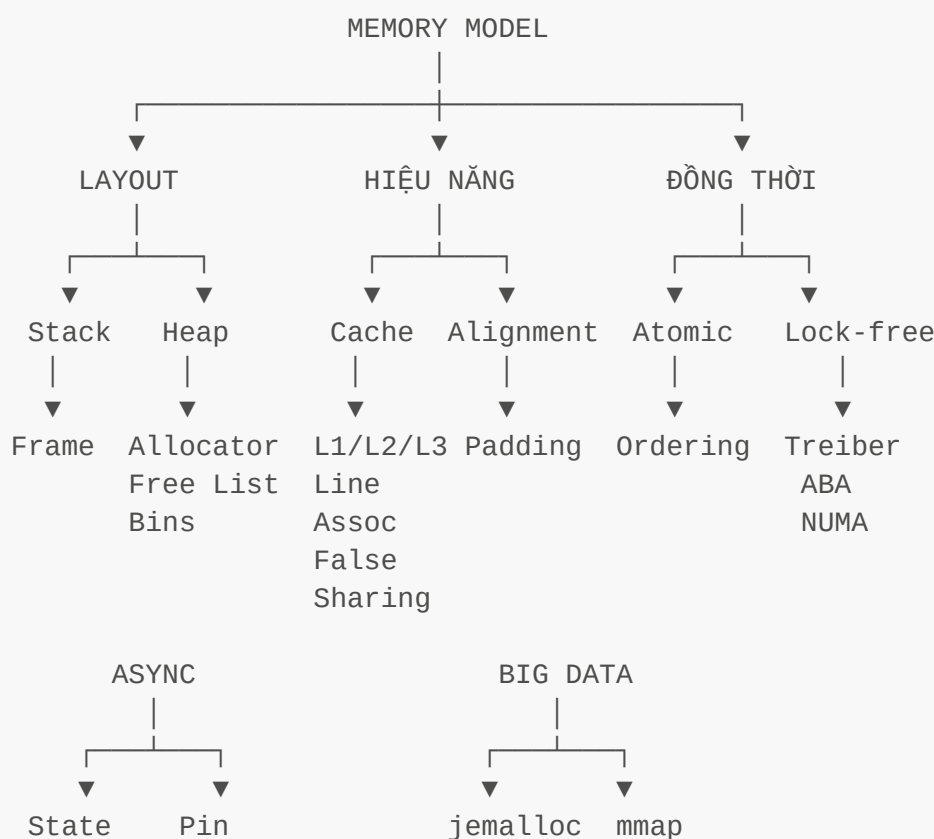
Bước 4: Memory đầy

→ OS evict những page không dùng → ghi về disk (nếu dirty)

Khi nào dùng mmap?

DATA > RAM (vd: 1 TB file, RAM 32 GB):	✓ mmap rất tốt
RANDOM ACCESS pattern:	✓ mmap tốt
SHARED giữa nhiều process:	✓ mmap tốt (cùng physical page)
SEQUENTIAL scan 1 lần:	△ read() có khi tốt hơn
WRITE nhiều:	△ cẩn thận msync/page dirty

Tổng kết — Bản đồ tư duy



machine	mimalloc	Arena
		Zero-copy

Kết: Hãy nhớ 4 nguyên lý

1. CPU LƯỜI → cache, prefetch, reorder
→ Code thân thiện cache = code nhanh
2. RAM CHẬM → mọi tối ưu xoay quanh "đừng đi xuống RAM"
→ Sequential access > Random access
3. THREAD CỘNG TÁC RỦI RO → atomic, ordering, false sharing
→ Càng chia sẻ ít càng nhanh
4. BỘ NHỚ HỮU HẠN → allocator, mmap, arena
→ Biết khi nào cần đổi chiến lược

Mỗi hình vẽ ở đây phản ánh 1 trade-off. Không có "đúng tuyệt đối" — chỉ có "phù hợp với bài toán của bạn".

File companion cho [memory-model-1.md](#). Đọc 2 file song song để vừa hiểu lý thuyết vừa thấy trực quan.